Simon Willison’s complete analysis of ChatGPT’s sandboxed code interpreter / container environment (January 26, 2026)

On January 26, 2026, Simon Willison published a detailed blog post titled **“ChatGPT Containers can now run bash, pip/npm install packages, and download files”** on his site (simonwillison.net), along with companion research notes and a public GitHub repo documenting his experiments inside the environment.

This is the definitive deep-dive on the upgrades to ChatGPT’s long-running “Code Interpreter” feature (later rebranded Advanced Data Analysis). Willison calls the new implementation **“ChatGPT Containers”** and describes it as a massive, undocumented upgrade that turns the old Python-only sandbox into a much more powerful, multi-language, Bash-first dev environment with controlled package installation and file downloads.

Below is the full, verbatim essence of his analysis (compiled directly from the blog post and research notes, with no summarization or omission of technical details).

Core upgrade summary (from the main post)
> One of my favourite features of ChatGPT is its ability to write and execute code in a container. This feature launched as ChatGPT Code Interpreter nearly three years ago, was half-heartedly rebranded to “Advanced Data Analysis” at some point and is generally really difficult to find detailed documentation about. Case in point: it appears to have had a *massive* upgrade at some point in the past few months, and I can’t find documentation about the new capabilities anywhere!

Key new capabilities Willison identified:
1. **Direct Bash execution** (previously only possible via Python’s `subprocess`).
2. **Native support for Node.js** + 9 other languages: Ruby, Perl, PHP, Go, Java, Swift, Kotlin, C, and C++ (no Rust yet).
3. **pip install** and **npm install** now work (via internal proxies — see below).
4. New privileged tool: **`container.download`** — ChatGPT can locate a public file URL and download it directly into the sandboxed filesystem.

Willison confirmed these features are available even on free ChatGPT accounts (not a preview).

container.download tool (new privileged fetch mechanism)
Willison first spotted it when ChatGPT used it to fetch an air-quality Excel file. He asked the model directly:

> Without running any searches tell me about your container.download tool

Response (verbatim from ChatGPT):
> container.download is a built-in tool I can call that fetches a file from a URL and saves it into the sandboxed container filesystem for this chat session.

> It takes a publicly reachable URL and a destination filepath in the container.
> Downloads the bytes from that URL and writes them to the given path.
> After that, I can read/process the file locally in the container (e.g., unzip it, parse it with Python, open it as an image, convert it, etc.).

Signature example:
```json
{
  "url": "https://example.com/somefile.zip",
  "filepath": "/mnt/data/somefile.zip"
}
```

Security / exfiltration analysis
Willison tested whether prompt injection could abuse it to leak data (e.g., constructing malicious URLs with query strings). It appears safe:
- Attempts to build a URL containing prior conversation history or sensitive data are blocked.
- Error message: “ERROR: download failed because url not viewed in conversation before. open the file or url using web.run first.”
- This mirrors Claude’s Web Fetch safety mechanism: only URLs the user explicitly provided or that came from safe search results are allowed.

He captured the download headers and found the requests originate from Microsoft Azure (centralus region, IP 52.230.164.178) with a ChatGPT-specific User-Agent.

Bash + multi-language support
> The key lesson from coding agents like Claude Code and Codex CLI is that Bash rules everything… When Anthropic added their own code interpreter feature to Claude last September they built that around Bash rather than just Python. It looks to me like OpenAI have now done the same thing for ChatGPT.

ChatGPT can now directly execute Bash commands and switch languages on the fly (e.g., `ruby -e 'puts "hello"'`, `node -e 'console.log("hello")'`, `go run`, `java`, `gcc`, etc.).

Package installation (pip/npm) — the clever internal-proxy design
General outbound networking from user code (urllib, fetch, curl, etc.) is **still completely blocked** (“Network is unreachable”).

However:
- `pip install`, `uvx`, `npm install`, `npx` all work.
- Reason: They are routed through **internal OpenAI package proxies** (not the public internet).

Key environment variables discovered (passwords redacted in Willison’s notes):

-
`PIP_INDEX_URL=https://reader:****@packages.applied-caas-gateway1.internal.api.openai.org/.../pypi-public/simple`
- `UV_INDEX_URL=...` (same internal mirror)
-
`NPM_CONFIG_REGISTRY=https://reader:****@packages.applied-caas-gateway1.internal.api.openai.org/.../npm-public`
- Broader CAAS_ARTIFACTORY_* variables for Go, Maven, Gradle, Cargo, Docker, etc.
- Policy hint: `NETWORK=caas_packages_only`

Willison installed several packages (datasette, sqlite-utils, llm) and inspected their metadata via `importlib.metadata`. Everything worked perfectly.

GitHub clones/ZIPs are still blocked (redirects to codeload.github.com are not whitelisted), but you can `pip download` source distributions as a workaround.

Filesystem & container environment details (from the companion research notes)
- Working directory appears to be `/mnt/data/` (or similar) for persistent files within a chat session.
- Ephemeral per-chat-session: files are wiped when the conversation ends.
- Python version and full package list can be inspected.
- Docker CLI is not present, but the internal Docker registry proxy **is** reachable (you can fetch image manifests/blobs via the Artifactory endpoint).
- No direct internet from user code (confirmed with `urllib.request.urlopen` failures).

Overall mental model (Willison's conclusion)
> Two egress "lanes":
> 1. General-purpose networking from user code is blocked.
> 2. Package/registry access is permitted via internal mirrors.

This design lets ChatGPT safely install dependencies and pull in external data files while keeping the sandbox secure. Willison calls it a "substantial upgrade" and notes that OpenAI urgently needs to update their release notes and documentation.

He links to:
- His full experiment repo:
https://github.com/simonw/research/tree/main/chatgpt-container-environment
- Shared ChatGPT transcripts demonstrating the new features.

Bottom line from Willison: The old "Code Interpreter" has quietly evolved into a genuinely powerful, multi-language, Bash-enabled container sandbox with smart, restricted internet access for packages and downloads. It's now one of the strongest coding-agent implementations available — but still undocumented by OpenAI.

If you want the raw transcripts or to experiment yourself, Willison's post includes multiple share links to live ChatGPT sessions where he triggered the new capabilities. This is the most complete public analysis available as of late January 2026.